

Investigating the use of Large Language Models (LLMs) in Test Case Generation

Leonardo Silva  [Universidade Federal do Rio de Janeiro | leonardocesar@ufrj.br]

André Gheventer  [Universidade Federal do Rio de Janeiro | gheventer@ufrj.br]

Patricia Gurgel  [Universidade Federal do Rio de Janeiro | patriciaamaralgurgel@ufrj.br]

Bruno Barroso  [Universidade Federal do Rio de Janeiro | blemosb@ufrj.br]

Lidvaldo dos Santos  [Universidade Federal do Rio de Janeiro | lidio@ufrj.br]

Rafael de Mello  [Universidade Federal do Rio de Janeiro | rafaelmello@dcc.ufrj.br]

Abstract

The increasing complexity of modern software systems and the pressure for shorter development cycles have intensified the need for more efficient verification practices. Test case generation remains a cornerstone of systematic and reproducible validation, yet manual [test case generation](#) continues to be costly, time-consuming, and prone to human error. In this context, Large Language Models (LLMs) have emerged as promising tools for automating the creation of functional test cases from user stories written in natural language. This paper reports a controlled study evaluating the performance of three LLMs on ten real user stories from a Brazilian public-sector enterprise system, comparing two prompting strategies: zero-shot and one-shot. Quantitative analysis showed comparable performance across strategies, with complementary strengths in precision and recall. A semantic analysis further revealed recurring defect types, including incorrect facts, ambiguities, inconsistencies, and omissions, which pose challenges for practical adoption. Despite these limitations, the findings demonstrate the feasibility of leveraging LLMs to support test case generation from user stories, while highlighting the need for expert oversight and improved prompt design.

Keywords: *Large Language Models, LLM, Software Testing, Test Case Generation, Quality Assurance*

1 Introduction

The increasing complexity of software systems and the demand for more agile development cycles have reinforced the importance of effective *Quality Assurance (QA)* practices. Among these, *Test Case Generation* plays a central role, as it ensures that the specified requirements are validated in a systematic manner (Myers et al., 2004; Whittaker, 2002). However, the manual creation of such artifacts remains a significant bottleneck in the [software development lifecycle \(SDLC\)](#). It is a costly activity, prone to omissions and inconsistencies, especially as the complexity of modern software outpaces traditional testing capabilities (Travassos et al., 2002).

Recently, Large Language Models (LLMs) have emerged as promising alternatives to automate software testing. However, integrating them into daily QA routines presents substantial challenges. As highlighted by Gheventer et al. (2026), despite the rapid proliferation of Generative AI tools, the field suffers from a “reliability gap”. Most current applications remain at an experimental stage and struggle with the contextual awareness needed for complex, real-world development environments. Furthermore, generative AI solutions for software quality are heavily focused on downstream implementation and verification activities, underscoring the need for studies that address the initial phases of SDLC.

Given this context, the present contributes to address these gaps by investigating the automated generation of test cases directly from user stories, guided by the following research questions:

- **RQ1:** To what extent can LLMs generate semantically correct test cases based on user stories?
- **RQ2:** To what extent do different LLMs (GPT-4o, DeepSeek, Gemini 1.5 Flash) differ in their effectiveness and consistency when generating test cases?
- **RQ3:** What is the effect of one-shot prompting compared to zero-shot on the effectiveness of LLMs in generating test cases?

To answer these questions, we conducted a controlled study using *ten user stories* from a real corporate system, assessing the ability of *three LLMs* to generate functional test cases based on two prompting strategies: *zero-shot* and *one-shot*. The investigation focuses not only on measuring LLMs’ accuracy, but also on identifying the incidence of semantic defects, adapted from the categorization of defects commonly applied in software inspections (Travassos et al., 1999).

This research provides relevant contributions to the field of *Quality Assurance (QA)* in software engineering, particularly regarding the application of *Large Language Models (LLMs)* to the automated generation of test cases from user stories. The main contributions are outlined below:

- *Experimental evaluation in a Real-World Scenario.* Design and execution of a controlled study using 10 real-world user stories from a complex industrial system. The study introduces a replicable framework that combines quantitative metrics (Precision, Recall, F1-Score) with a qualitative defect analysis, ensuring high methodological rigor and external validity.

- *Identification of Omission as the Core Limitation.* The study empirically identifies omission as the most frequent and critical failure mode of LLMs in test case generation, far surpassing identified defects. This highlights a fundamental challenge in achieving full requirement coverage based on implicit criteria.
- *Comparative Analysis of LLMs and Prompting Strategies.* The study provides a rigorous statistical and qualitative comparison, revealing: (a) a performance plateau with no statistically superior model for this task, and (b) a clear trade-off where one-shot prompting improves precision while zero-shot favors recall, with no significant difference in overall effectiveness (F1-Score).
- *Practical Implications for QA Processes.* The findings provide actionable insights for the industry, including redefining the QA analyst's role from "creator" to "context provider and curator" and emphasizing that the choice of LLM tool can be guided by practical criteria (cost, latency) rather than marginal performance differences.

The results contribute to a critical understanding of the role of LLMs in test artifact generation, providing evidence of their accuracy, limitations, and error patterns (Runeson et al., 2012). Furthermore, they offer insights for researchers and practitioners in Software Engineering interested in exploring the integration of generative AI into QA activities, as well as in improving the generation of test cases and troubleshooting in software performance testing (Verkholomova and Nazaruka, 2025).

The remainder of this paper is structured as follows: Section 2 reviews the related work. Section 3 details our experimental design. Section 4 presents the experimental results, and Section 5 discusses the threats to their validity. Finally, Section 6 interprets the implications of our findings, and Section 7 concludes the paper.

2 Related Work

In the context of LLM-based test automation, one stream of research investigates the generation of unit tests directly from source code. Schäfer et al. (2023) present a large-scale empirical evaluation of LLMs' effectiveness in automated unit test generation without manual effort. Their results indicate that the effectiveness of the testing approach is influenced by the size and training set of the LLM. Similarly, Chen et al. (2024) proposed an LLM-based automated unit test generation framework. Their results demonstrated greater code coverage in half of the evaluated projects. Wang et al. (2024) complemented this line of work by introducing a method that decomposes complex Java methods into slices and uses LLMs to generate tests for each slice. The results showed that the method outperformed current test-generation methods in both line and branch coverage.

Beyond unit testing, another stream of research focuses on generating test artifacts directly from natural language specifications. Santos et al. (2024) employed an LLM-based tool to automatically generate tests directly from user stories and

acceptance criteria structured via Behavior-Driven Development (BDD). Their approach showed a significant increase in test coverage. Furthermore, Mendoza et al. (2024) evaluated the understanding of BDD scenarios for automated test data generation, classifying them by complexity. While these studies successfully translate requirements into QA artifacts, their evaluations rely strictly on structural code coverage or subjective assessments.

Finally, Li et al. (2025) conducted a comprehensive empirical evaluation of LLMs applied to broad software testing tasks, including test case generation, error tracing, and bug localization on open-source projects. Their findings emphasize that while LLMs can effectively complement manual testing, they suffer heavily from hallucinations, frequently producing irrelevant or factually incorrect outputs. This highlights a critical need for human oversight and rigorous validation methods beyond mere execution rates or bug detection capabilities. Table ?? presents a comparative overview of these works. Unlike existing research that focuses on code-centric generation or evaluates requirement-to-test translation using structural metrics, our study addresses the reliability gap by assessing the semantic correctness of functional specifications through the application of a formal defect taxonomy combined with effectiveness metrics (Precision, Recall, and F1-score) validated against a human-expert oracle.

3 Study Design

This section describes the design of the controlled study evaluating the effectiveness of Large Language Models (LLMs) in generating test cases from user stories. The study setup was carefully planned to yield reliable, reproducible results representative of the challenges test teams face in agile, real-world scenarios. The complete replication package is publicly available on GitHub (<https://github.com/leonardocesarc/testcase>). It includes the dataset of user stories, the full set of original prompts, and the generated test cases. To preserve the semantic validity of the analysis, the textual artifacts are maintained in their original language (Portuguese), accompanied by an English README file and standardized spreadsheets with English metadata for accessibility.

The study was designed to systematically answer the research questions presented in the Introduction, comparing different models and prompting strategies under controlled conditions to ensure the reliability and validity of the findings.

3.1 Research Questions and Hypotheses

To guide our investigation, the study was structured around four central research questions (RQs). Each question led to the formulation of a null hypothesis (H_0), which posits the absence of a statistically significant effect or difference, and an alternative hypothesis (H_1), which posits its existence. Table 2 presents the direct relationship between the research questions and the formulated hypotheses, which were subsequently tested through statistical analysis.

Table 1. Comparison of Related Works

Study	Input Artifact	Generated Output	Evaluation Focus	Main Differences
<i>This Study</i>	User Stories	Functional Test Cases	Semantic Correctness	<i>Evaluates semantic correctness directly from requirements using a defect taxonomy.</i>
Schäfer et al. (2023)	Source code	Unit tests	Code coverage	<i>Code-centric approach; relies on structural metrics rather than semantic alignment.</i>
Chen et al. (2024)	Source code	Unit tests	Code coverage	<i>Code-centric approach; focuses on implementation verification rather than requirements.</i>
Wang et al. (2024)	Java methods	Unit tests	Line and branch coverage	<i>Decomposes complex code, but remains strictly bound to structural code metrics.</i>
Santos et al. (2024)	User stories and BDD criteria	Automated BDD acceptance tests	Test coverage metrics	<i>Translates requirements, but evaluates success strictly through structural code coverage.</i>
Mendoza et al. (2024)	BDD scenarios in Gherkin	Test data generation	Structural coverage or subjective assessment	<i>Focuses on test data rather than scripts, without formal semantic validation of logic.</i>
Li et al. (2025)	Open-source projects (Code/Bugs)	Test cases, bug localization	Execution rates and bug detection capabilities	<i>Broad empirical evaluation highlighting hallucination frequency compared to manual testing.</i>

Table 2. Relationship between Research Questions and Hypotheses.

RQ	Hypotheses
RQ1	H_0 LLMs generate test cases with a low level of semantic correctness and a high frequency of defects. H_1 LLMs generate test cases with a high level of semantic correctness and a low frequency of defects.
RQ2	H_0 There is no significant difference in effectiveness and consistency among the evaluated models (GPT-4o, DeepSeek, Gemini 1.5 Flash). H_1 There are significant differences in effectiveness and consistency among the models.
RQ3	H_0 There is no significant difference in the effectiveness (measured by F1-Score) of test case generation between the zero-shot and one-shot prompting strategies. H_1 There is a significant difference in the effectiveness of test case generation between the prompting strategies.

3.2 Variables

The experiment was designed with a clear set of independent and dependent variables to isolate effects and measure outcomes objectively.

3.2.1 Independent Variables

The following two factors were systematically manipulated during the experiment:

- **LLM Model:** Three Large Language Models (LLMs) were selected for the experiment, representing distinct architectures and design approaches:
 - **GPT-4o (OpenAI):** A multimodal Transformer model widely used for reasoning and context-aware text generation.
 - **DeepSeek (Hangzhou DeepSeek AI Co., Ltd):** A lightweight model optimized for efficiency and cost reduction, combining supervised learning with reinforcement learning from human feedback (RLHF).
 - **Gemini 1.5 Flash (Google DeepMind):** A multimodal model featuring advanced symbolic and contextual reasoning, optimized for fast inference and long-context processing.
- **Prompting Strategy:** Two classical prompting techniques were applied to the selected LLMs (Brown et al., 2020):
 - **zero-shot:** The LLM was asked to generate test cases based solely on the user story and a set of instructions, without any prior examples.

- one-shot: The prompt included a complete, illustrative example of a test case before the target user story, serving as a structural and semantic reference for the generation task.

3.2.2 Dependent Variables

To evaluate the LLMs' performance, the following dependent variables were measured:

- **Quantitative Metrics:** To assess the accuracy and coverage of the generated test cases, we measured *Precision*, *Recall*, and *F1-Score*. These metrics were calculated by comparing the LLM-generated outputs against the human-created reference oracle.
- **Semantic Quality Criteria:** To assess the semantic quality of the generated test cases, we manually analyzed and classified each output based on criteria adapted from (Travassos et al., 1999). This process involved: (i) categorizing semantic defects in incorrect test cases, including *Incorrect Fact* (factually wrong information), *Ambiguity* (vague or multi-interpretable statements), and *Inconsistency* (a contradiction between steps, acceptance criteria, or contextual expectations); (ii) annotating the presence of *Extraneous Information* (elements in a generated test case that are irrelevant to the source user story) in correct test cases; and (iii) systematically recording *Omissions* (the absence of expected test cases) by comparing the generated set against the human oracle.

3.3 Dataset and Reference Oracle

The dataset used in this study was extracted from a real world enterprise system used by a Brazilian judicial body.

The choice enterprise system was motivated by the need to evaluate the LLMs' performance in a practical, industry relevant scenario judicial. The dataset consists of ten real user stories (USs) in Portuguese, detailed in Table 3, that describe distinct system functionalities. For each user story, a corresponding set of manual test cases had been previously created by a team of experienced software testing specialists. These manual test cases were extracted directly from the official QA documentation of the enterprise system, representing production validated acceptance criteria rather than synthetic academic examples. This human generated set of test cases served as the oracle for our experiment, establishing the ground truth for evaluating the correctness and completeness of the LLM generated outputs.

The user stories were selected to ensure functional diversity, covering a range of system behaviors from simple data entry to more complex business rules and user interactions. Due to the exploratory nature of this study and the need for in-depth qualitative analysis of the generated test cases, a subset of ten user stories was intentionally selected from a larger available pool. All sensitive information was anonymized to protect confidentiality. All user stories used were carefully anonymized and reviewed by domain experts to ensure relevance, pertinence to the research context, and

Table 3. User Stories used in the experiment

ID	User Story (summary)
US01	Automatically save quotes and initial petitions
US02	Acknowledge documents through the process tree
US03	Activate or deactivate subjects in the system
US04	Modify the dossier record interface for improved visualization
US05	Display process identifier with last movement
US06	Access protective forms through modal windows
US07	Display useful links section
US08	Include confidentiality between agencies in the "Child Protection" tab
US09	Access "Social Profile and Family Contact" tab
US10	Access "Current Shelter" tab in the protective form

varying degrees of functional complexity, ranging from simple workflows to conditional and multi-step business rules. To provide a concrete illustration of the input artifacts provided to the LLMs, Figure 1 presents a representative (and anonymized) example of a user story from the dataset.

Title: US01 - Automatically save quotes and initial petitions

User Story:

As a writer of quotas and petitions, I would like them to be saved automatically to avoid data loss, for example, due to timeouts or connection problems.

Acceptance Criteria (ACs):

1. **AC1:** When creating a quota or initial petition, the system should provide a tool to automatically save the data entered by the user;
2. **AC2:** The saving should occur every X minutes (or characters typed?).
3. **AC3:** If the user comes from the notifications box, and there is an automatically saved notification different from the notification in the box, do not overwrite the notification field with the automatically saved data.
4. **AC4:** Creation of movement and required data.

Figure 1. Example of a user story used in the experiment, including its title, narrative, and acceptance criteria.

3.4 Prompt Design Strategy

We adopted a structured prompting approach grounded in the prompt engineering patterns proposed by White et al. (2023). Specifically, we employed the *persona pattern* to establish the behavior of a Senior QA Specialist, requiring knowledge of ISTQB standards (see Figure 2), and the *context manager pattern* to bound the scope of the generation. The instructions explicitly directed the models to apply Equivalence Partitioning and followed strict output formatting. This included requiring specific fields (ID, Name, Pre-condition, Objective, and Expected Result) while forbidding conversational filler. For the one-shot strategy, a concrete example of a login validation test case was appended. It is important to highlight that the user stories (input data) were extracted from a Brazilian governmental system used by public defenders to provide legal guidance to citizens. As such, the original requirements were written in Portuguese and contain terminology and jargon specific to the Brazilian legal domain. Therefore, to preserve semantic nuances and avoid potential noise introduced by translation layers, all prompts were constructed and processed in the original language.

TESTER ROLE SKILLS: I want you to act as a testing specialist with experience in the field of Software Engineering, specifically in the discipline of Software Testing. It is necessary that you know the content of the Certified Tester Foundation Level (CTFL) certification from the International Software Testing Qualifications Board (ISTQB). Your main responsibility will be to create test cases, demonstrating advanced technical skills.

Figure 2. Definition of “Tester Skills” used in the Prompt (Translated from Portuguese).

To structure the interaction with the LLMs, a standardized Prompt Prefix was systematically applied before each user story. This prefix was derived directly from the experimental protocol and consisted of five components:

(1) *Context and Guidelines*, defining the core task of generating test cases from user stories written in natural language. The user stories followed a three-part structure: (i) title, (ii) description using the standard format (“I, as a... I would like... in order to...”), and (iii) acceptance criteria. The models were explicitly instructed to prioritize the functional description over acceptance criteria when deriving test cases, ensuring that test case generation was driven by requirements rather than validation statements. Additionally, the prompt emphasized the use of the equivalence partitioning technique to systematically explore variations in inputs, system states, and behaviors, promoting more effective and representative test coverage. The models were also instructed to avoid making assumptions about unspecified requirements, reinforcing controlled and faithful interpretation of the input;

(2) *Tester Role Definition*, assigning the model the role of a Software Testing Specialist with expertise aligned with the ISTQB Certified Tester Foundation Level (CTFL). This role-playing strategy was intended to guide the model toward producing technically sound and professionally structured test cases, reflecting established testing principles and best practices in Software Engineering;

(3) *System Domain Description*, providing a detailed contextualization of the target system. The domain corresponds to a large-scale information system used by a Brazilian Public Defender’s Office, involving high-volume daily operations, integration with judicial and governmental systems, and multiple specialized modules. This contextual grounding supports the generation of realistic and domain-relevant test cases, particularly in scenarios involving complex workflows and institutional constraints;

(4) *Output and Formatting Guidelines*, specifying strict requirements for the structure and content of the generated test cases. Each test case was required to include: (i) identifier (CTn), (ii) name, (iii) preconditions (if applicable), (iv) objective, and (v) expected result. The models were instructed to produce concise outputs, avoid including procedural test steps, maintain consistent formatting across all generated cases, and ensure sequential numbering restarting for each user story. Furthermore, the interaction protocol required the model to process one user story at a time and signal readiness for the next input, ensuring controlled execution;

(5) *Test Case Model Output* (One-shot only), included exclusively in the one-shot configuration. In this setup, the prompt was augmented with a single illustrative example of

a test case, demonstrating the expected structure, level of detail, and formatting conventions. This example served as a reference pattern to guide the model’s output generation, reducing ambiguity and improving consistency. In contrast, the zero-shot configuration did not include any example, requiring the model to rely entirely on the instructions and its pre-trained knowledge.

To ensure the statistical independence of results and eliminate potential carryover effects, each execution was performed in a fresh session, resetting the model’s context window. This procedure guarantees that each generated output represents an independent inference, unaffected by previous interactions.

Finally, two prompting strategies were applied. In the zero-shot configuration, the LLM generated test cases without any prior examples, relying solely on the prompt instructions. In the one-shot configuration, the same prompt was extended with a single model output example (TEST CASE MODEL OUTPUT), enabling the evaluation of how minimal demonstration influences the quality, consistency, and structure of the generated test cases.

The prompts were intentionally designed to allow the LLM flexibility in generating test cases, rather than enforcing a fixed or minimum number per user story. This decision aligns with the research objective of observing how LLMs behave under less constrained conditions, simulating a more realistic and exploratory test case generation scenario. Therefore, no explicit instruction was included in the prompts requiring the generation of at least one test case per user story, nor defining an upper-bound limit. The number of test cases produced per run (averaging 8.5) emerged naturally from the model’s interpretation of each user story’s complexity and scope, in accordance with the agreed experimental design and research goals, which prioritized variability and autonomy over rigid output constraints.

3.5 Experimental Procedure

The experiment followed a systematic, multi-stage procedure to ensure the reliability and reproducibility of the findings. The complete workflow, from the initial input artifacts and prompting strategies through generation, post-processing, and evaluation, is visually summarized in Figure 3. As depicted, for each of the 10 user stories, both zero-shot and one-shot prompting strategies were applied three times across the three selected LLMs, resulting in a total of 180 runs. This entire process was conducted between June and September 2025, using the official web interfaces of the selected LLMs¹. The core generation phase consisted of the following steps for each of the ten user stories in the dataset:

1. **Input Prompting:** For each user story, two distinct prompts were prepared based on the design defined in Section 3.4: one for the zero-shot strategy and another for the one-shot strategy. The prompts were carefully standardized to maintain consistency, varying only in the inclusion of the illustrative example for the one-shot

¹ChatGPT (<https://chat.openai.com>), DeepSeek (<https://www.deepseek.com>), and Gemini (<https://gemini.google.com>).

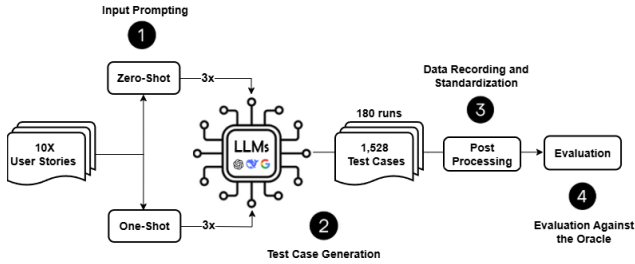


Figure 3. Overview of experimental workflow.

condition. To reach the final version, the prompt instructions and formats were iteratively refined based on the results of a pilot run. This pilot was conducted using a single user story not included in the final experimental dataset. Throughout this refinement process, the researchers reviewed multiple versions of each prompt to enhance clarity and minimize ambiguity.

2. **Prompt Refinement (Pilot Study):** Before the main execution, the prompts underwent an iterative refinement process. First, we conducted a pilot run using a single user story, which was not included in the final experimental dataset. This step aimed to validate the clarity of the instructions (Persona, Context, and Task) and ensure the output format was machine-readable. Based on the pilot results, we made minor adjustments to the phrasing to minimize ambiguity before applying the prompts to the 10 target user stories. Before the main execution, the prompts underwent an iterative refinement process.
3. **Test Case Generation:** Each of the three LLMs (GPT-4o, DeepSeek, Gemini 1.5 Flash) was fed with the prompts from the previous step. This process was independently repeated three times for every combination of user story, LLM, and prompting strategy, resulting in 180 execution runs (10 USs $\times$ 3 LLMs $\times$ 2 strategies $\times$ 3 repetitions) and a total of 1,528 generated test cases for analysis. The multiple repetitions allowed for the analysis of the models' output variability, while the controlled conditions ensured that any observed differences could be attributed to the independent variables.
4. **Data Recording and Standardization:** The 1,528 generated test cases were recorded in a spreadsheet, and a manual post-processing step was performed to standardize formatting and isolate the outputs for evaluation.
5. **Evaluation Against the Oracle:** To perform the evaluation, the generated test cases were analyzed against the human created ground truth by a panel of three reviewers to mitigate individual bias. The panel consisted of two postgraduate researchers (Master's and Ph.D. candidates) with experience in software engineering, and a senior researcher with extensive background in software testing and experimental methods. Initially, the generated artifacts were independently evaluated based on the predefined defect taxonomy (Travassos et al., 1999). In a subsequent stage, a double-check process was performed across the dataset to cross-validate the classifications. Any disagreements between the two postgraduate researchers were discussed and resolved through consensus, with the senior researcher acting as the media-

tor and final arbiter. Based on this validated comparison process, each case was classified as either Correct or Defective. Correct cases were further annotated if they contained *Extraneous Information*. Defective cases were categorized into one of the qualitative defect types: *Incorrect Fact*, *Ambiguity*, or *Inconsistency*. *Omissions* were identified by cross-referencing the oracle to find expected test cases that were not generated.

3.6 Data Analysis

The data collected from the 180 execution runs were analyzed through both qualitative and quantitative approaches. The following subsections detail the criteria, metrics, and statistical tests used for this analysis.

3.6.1 Semantic Evaluation Criteria

Based on artifact inspection techniques in software engineering (Travassos et al., 1999), we defined a set of criteria to assess the semantic quality aspects of the generated test cases.

Definition of Correctness: The notion of correctness adopted in this study follows a strict functional criterion rather than exact textual matching. A generated test case is classified as Correct (True Positive) if its pre-conditions, steps, and expected results accurately cover the acceptance criteria of the user story. We adopted an "all-or-nothing" judgment regarding semantic validity, where if a multi-step test case contains a single Incorrect Fact, Inconsistency, or Ambiguity, the entire artifact is deemed Defective (False Positive). A test case is still considered Correct if it includes Extraneous Information (e.g., additional but valid steps), provided this extra data does not compromise the test's logical execution.

To operationalize this classification, the specific categories are defined as follows:

- **Incorrect Fact:** Presence of factually wrong information that contradicts the user story's acceptance criteria or domain knowledge.
- **Inconsistency:** Presence of contradictions between steps, acceptance criteria, or contextual expectations within the test case.
- **Ambiguity:** Presence of vague or multi-interpretable textual elements that could hinder test execution.
- **Extraneous Information:** Inclusion of elements irrelevant to the domain or the user story. This was annotated but not treated as a defect.
- **Omission:** Absence of a test case that was expected according to the human reference oracle.

3.6.2 Quantitative Analysis

To measure the models' effectiveness, we used standard metrics based on the components of a confusion matrix. For the context of this study, a *True Positive (TP)* was defined as a generated test case classified as "Correct." A *False Positive (FP)* was a test case classified as "Defective" (due to Incorrect Fact, Inconsistency, or Ambiguity), and a *False Negative*

(FN) was an expected test case from the human oracle that the LLM lacks on generating (omission).

Based on these components, the following three metrics were calculated for each execution run:

- **Precision:** The proportion of correctly generated test cases (TP) among all generated cases (TP + FP). It measures the accuracy of the generated outputs.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** The proportion of correctly generated test cases (TP) among all expected test cases from the oracle (TP + FN). It measures the coverage of the model.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score:** The harmonic mean of Precision and Recall, providing a single score that balances both metrics.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Handling of Repeated Runs: To account for the non-deterministic nature of LLMs, each user story was executed three times per model and prompting strategy. For the statistical analysis, these repetitions were aggregated by user story. Specifically, we calculated the mean performance across the three runs, resulting in 10 independent observations per configuration.

To test our hypotheses with statistical rigor, we adopted a significance level of $\alpha = 0.05$. Given the data distribution, the following non-parametric tests were employed:

- The *Friedman test* (Friedman, 1937) was used to compare the F1-Score performance across the three LLM models (RQ2).
- The *Wilcoxon signed-rank test* (Wilcoxon, 1945) was applied separately to the results of Precision, Recall, and F1-Score to compare the effectiveness of the paired prompting strategies (RQ3).

4 Results

In this section, we present the results of the experiment conducted to evaluate the capability of three Large Language Models (GPT-4o, DeepSeek, and Gemini 1.5 Flash) in generating test cases. The analysis is based on a total of 1,528 test cases generated across 180 executions, covering ten distinct user stories under two prompting strategies (zero-shot and one-shot).

4.1 Overall Quality of Generated Test Cases

The qualitative assessment revealed that a substantial majority of the generated test cases, 1,110 (72.68%) of the total, were judged as correct. Conversely, 418 cases (27.38%) were classified as defective. Table 4 provides a detailed breakdown of these categories. Among the defects, *Incorrect Fact*

was the most frequent category, accounting for 12.04% of all generated cases, followed by *Ambiguity* (8.70%) and *Inconsistency* (6.61%). This distribution suggests that the most significant challenge for LLMs generating effective test cases lies in adhering to domain and requirement specifications, rather than in maintaining internal logical cohesion.

Table 4. Overall Classification of Generated Test Cases (N=1,528)

Category	Count	Percentage (%)
Correct	1,110	72.68
Without Extraneous Information	819	53.61
With Extraneous Information	291	19.06
Defective	418	27.38
Incorrect Fact	184	12.04
Ambiguity	133	8.70
Inconsistency	101	6.61
Total	1,528	100.00

To provide a deeper answer to RQ1, our analysis focused on two critical aspects of generation quality: the presence of defects in the generated test cases (quantified by False Positives, FP) and the omission of expected test cases (quantified by False Negatives, FN).

As summarized in the Table 5, the analysis revealed a key trade-off between the prompting strategies. The zero-shot approach was more prone to producing incorrect outputs, resulting in a higher False Positive count (234 FP) than one-shot (184 FP). This finding suggests that, without an example, models are more likely to generate irrelevant or factually incorrect test cases.

Conversely, the one-shot approach was more likely to omit expected outputs. It resulted in a higher number of False Negatives (543 FN) compared to zero-shot (508 FN). This finding highlights a critical qualitative insight: while providing an example (one-shot) improves the quality of the generated content, it may also constrain the model, causing it to overlook valid test cases. This tendency to omit relevant tests emerged as the most significant challenge to the overall quality.

Table 5. Distribution of Defects and Omissions by Prompting Strategy

Strategy	FP	(%)	FN	(%)
zero-shot	234	31,54	508	68,46
one-shot	184	25,31	543	74,69

Although zero-shot favors recall (68.47% FN) by avoiding excessive constraint to the example, while one-shot increases omissions (74.70% FN), mitigation strategies include: (i) prompts enhanced via iterative refinement, as performed in the pilot study (Section 3.5); (ii) hybrid pipelines with human post-generation validation against reference oracle; (iii) contextualized prompt design with explicit system domain description (Section 3.4). These adjustments, aligned with the prompt engineering practices already employed, reinforce the practical viability of LLMs for assisted test case generation in agile QA.

Summary of findings for RQ1

LLMs demonstrated a high level of overall correctness (72.68%), yet their quality is dichotomous. They are effective at generating test scenarios from explicit requirements but systematically fail to address context-dependent issues, leading to critical omissions and incorrect facts. Given the high precision rate, which contradicts the premise of low performance, we *reject the null hypothesis (H_0)* for RQ1.

4.2 Performance Comparison Among LLM Models

Table 6 presents the mean values for the primary performance metrics, precision, recall, and F1-Score, for each LLM under both prompting strategies. The results provide a comprehensive overview of the models' performance before statistical testing. A few key trends emerge from this data. First, one may see a consistent trend across all models and prompting strategies in reaching considerably higher precision than recall. In other words, there is a general tendency of LLMs to generate correct test cases (high precision), while they struggle with covering all expected scenarios (low recall). Second, a clear trade-off between the prompting strategies is visible: the one-shot strategy tends to slightly increase precision at the cost of a more noticeable decrease in recall when compared to zero-shot. Finally, while GPT-4o consistently achieved higher F1-scores, the overall performance differences among the models appear to be minor.

Table 6. Mean Performance Metrics by LLM and Prompting Strategy

Model	Precision	Recall	F1-Score
<i>zero-shot Strategy</i>			
GPT-4o	0.728	0.594	0.645
DeepSeek	0.736	0.572	0.628
Gemini 1.5 Flash	0.689	0.525	0.581
Overall Mean	0.718	0.564	0.618
<i>one-shot Strategy</i>			
GPT-4o	0.767	0.539	0.618
DeepSeek	0.700	0.527	0.590
Gemini 1.5 Flash	0.757	0.463	0.553
Overall Mean	0.741	0.510	0.587

To answer RQ2, which investigates whether there is a significant performance difference among the evaluated models, we conducted a statistical analysis on the F1-Scores presented in our overall performance summary (Table 6). The descriptive data in that table reveals minor variations, with GPT-4o consistently achieving the highest F1-Score in both the zero-shot (0.645) and one-shot (0.618) scenarios. The performance range between the highest- and lowest-performing models was nearly identical in both conditions (0.064 for zero-shot and 0.065 for one-shot), suggesting a similar level of performance.

To determine if these numerical differences were statistically significant, we applied the *Friedman test*. As detailed

in Table 7, the analysis found no statistically significant difference in performance among the LLMs under either condition. For the zero-shot strategy, the result was $Q = 2.400$ ($p = 0.3012$), and for one-shot, it was $Q = 3.200$ ($p = 0.2019$). As the p-values were substantially higher than our significance level of $\alpha = 0.05$, we *fail to reject the null hypothesis (H_0)* for RQ2.

While we could not identify significant differences among models in overall effectiveness, a closer inspection of the F1-Score distributions, as shown in Figure 4, reveals distinct performance profiles. The boxplots illustrate that while the models are comparable in their median performance, they differ in their variability and predictability. This finding reinforces that for the task of generating test cases from our dataset, the three evaluated models performed.

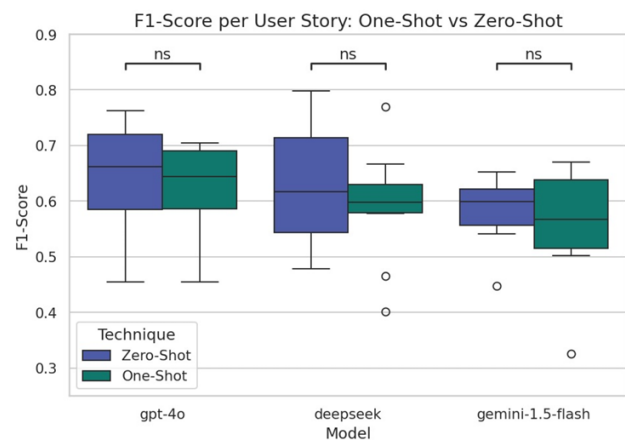


Figure 4. Distribution of F1-Scores per User Story, comparing zero-shot and one-shot techniques across the three LLMs.

Table 7. Results of the Friedman Test for Comparison Among LLMs

Technique	Q-statistic	p-value
zero-shot	2.400	0.3012
one-shot	3.200	0.2019

Summary of findings for RQ2

Despite minor variations in performance profiles, our statistical analysis concluded that there was no significant difference in effectiveness among the three LLMs tested. Consequently, we *fail to reject the null hypothesis (H_0)* for RQ2.

4.3 Impact of Prompting Strategies: zero-shot vs. one-shot

To answer RQ3, we analyzed the impact of the prompting strategy on model performance by comparing the results between the zero-shot and one-shot conditions. The descriptive statistics in our overall performance summary (Table 6) reveal a clear trade-off: the one-shot strategy consistently achieved a higher average precision, whereas the zero-shot approach yielded a superior recall across all models.

This trade-off is further illuminated by the distribution of defects and omissions. As detailed in Table 5, the one-shot approach produced significantly fewer false positives (184 FP) compared to zero-shot (234 FP). However, this improvement in accuracy was counterbalanced by a higher number of omissions, with one-shot resulting in more false negatives (543 FN) than zero-shot (508 FN).

To determine whether these trade-offs resulted in a statistically significant difference in overall effectiveness (F1-Score), we applied the *Wilcoxon Signed-Rank test*. The results, presented in Table 8, confirm that there was no statistically significant difference between the two strategies for any of the three LLMs. With p-values for GPT-4o ($p = 0.5280$), DeepSeek ($p = 0.1309$), and Gemini 1.5 Flash ($p = 0.4316$) all substantially higher than our significance level of $\alpha = 0.05$, we *fail to reject the null hypothesis* (H_0) for RQ3. Furthermore, the small median differences and effect sizes (r) reinforce that the observed variations are not statistically meaningful.

Figure 4 illustrates this statistical equivalence. The distribution of F1-Scores shows that the performance medians are remarkably close for both techniques across all models. The outliers observed, which represent instances of particularly low performance, typically correspond to executions on a small subset of more complex user stories, where models struggled to interpret all acceptance criteria, thus resulting in a lower F1-Score.

Table 8. Results of the Wilcoxon Signed-Rank Test (zero-shot vs. one-shot per LLM)

Model	Median Diff.	W	p-value	Effect Size (r)
GPT-4o	0.035	12.0	0.5280	0.364
DeepSeek	0.032	12.0	0.1309	0.478
Gemini 1.5 Flash	0.034	19.0	0.4316	0.249

Each box plot represents the distribution of F1-Scores from the 10 user stories. The central line indicates the median, the box represents the interquartile range (IQR), and the whiskers extend to 1.5 times the IQR. Outliers are shown as individual points. The ‘ns’ denotes a non-significant difference.

Summary of findings for RQ3

Our statistical analysis concluded that there was no significant difference in overall effectiveness (F1-Score) between the zero-shot and one-shot prompting strategies. Besides, we observe a trend of: one-shot improved precision, while zero-shot yielded higher recall. Therefore, we *fail to reject the null hypothesis* (H_0) for RQ3.

They reliably identify test scenarios based on explicit requirements but struggle with omissions that require deeper contextual inference. Furthermore, our analysis reveals a performance plateau, with no statistically significant difference found among the evaluated models. Similarly, the choice of prompting strategy was found to influence the trade-off between precision and recall but did not result in a significant difference in overall effectiveness. The implications of these

findings will be discussed in the Discussion section.

5 Threats to validity

This section discusses potential threats that could influence the interpretation and generalization of the results. Following the guidelines of (Runeson et al., 2012), the discussion considers four dimensions of validity— internal, external, construct, and statistical —specific to studies involving Large Language Models (LLMs) in software testing contexts.

5.1 Internal Validity

Internal validity concerns whether observed outcomes can be attributed to the intended experimental factors— namely, the prompting strategy and the LLM model —rather than to uncontrolled variables. Potential threats include:

- limited variety of user stories (selection bias);
- inherent non-determinism of LLMs: ChatGPT can exhibit high run-to-run variation in code generation (Ouyang et al., 2025);
- inconsistencies in manual assessment of test cases.

Mitigation: We composed a sample with user stories varying in complexity and diversity of functionalities. The study execution followed standardized prompts and replicated runs per setting, and we monitored the whole execution steps behavior across runs. Besides, multiple reviewers cross-checked manual evaluations. However, it remains possible that minor effects from model drift may influence the results.

5.2 External Validity

External validity addresses the extent to which the findings can be generalized beyond the analyzed context (Wohlin et al., 2012). The experiment employed user stories from a single enterprise system written in Portuguese, which limits transferability to other domains, organizations, or languages. However, the use of production-grade artifacts from a real environment enhances external validity (Verdecchia et al., 2023), offering a realistic scenario for assessing LLM-based test generation. To broaden generalizability, future replications should include multiple datasets from different projects, and compare prompting strategies (zero-shot vs. one-shot), since these factors may influence accuracy and coverage patterns (Chen et al., 2024).

Mitigation: A practical and exploratory approach was adopted to assess the feasibility of applying LLMs in real-world development contexts. We thoroughly documented all methodological decisions to facilitate replication, adaptation, or extension in future studies. Prior work on LLM-based test generation frequently highlights this limitation, as many studies rely on academic datasets or simple benchmarks with limited domain or language variety (Celik and Mahmoud, 2025).

5.3 Construct Validity

Construct validity concerns whether the adopted measures accurately capture the intended concepts of test, quality, and

semantic correctness. In this study, we adopted a defect taxonomy from (Travassos et al., 1999), which defines these categories as recurring defect types in software artifacts and guides human-centered evaluations during inspection. Despite calibration sessions among evaluators, the qualitative analysis of defects involves subjective judgment, which may have influenced individual ratings (Fernández et al., 2017).

Mitigation: To minimize this threat, the evaluation criteria were defined a priori and consistently applied across all samples. Independent reviewers cross-checked classifications, and disagreements were resolved by consensus. Employing the well-established taxonomy of (Travassos et al., 1999) helped maintain semantic consistency across categories. Nonetheless, minor interpretation differences between reviewers may have affected marginal cases.

5.4 Statistical Conclusion Validity

This validity concerns the confidence that the observed differences in results are not due to chance but represent real effects (Wohlin et al., 2012). It addresses the reliability of statistical inferences drawn from the data and the extent to which they reflect true relationships between treatments and outcomes. Potential threats include the limited sample size, violations of statistical assumptions, multiple comparisons, and the associated risks of Type I (false positive) and Type II (false negative) errors. *Although the experiment involved ten user stories, each was executed three times per model-prompting combination, yielding 180 valid generations. To ensure robust statistical testing and avoid pseudoreplication, these runs were aggregated into 10 independent observations per approach (zero-shot and one-shot). This design enabled the application of statistical testing but still carries the inherent risk of overinterpreting random variation, especially with small effect sizes.*

Mitigation: To address these risks, non-parametric tests were used, along with effect-size calculations and confidence intervals, which are less sensitive to violations of normality and appropriate for small samples. Consistent procedures, controlled repetitions, and the documentation of all runs helped improve reliability and minimize random heterogeneity. Nevertheless, the statistical findings should be interpreted as indicative rather than conclusive, considering the stochastic nature of LLM outputs and the constraints on statistical power.

Overall, the methodological controls adopted, including standardized prompts, balanced runs, replicated evaluations, and triangulation between quantitative and qualitative evidence, contribute to the consistency of the findings. Nevertheless, as emphasized in (Bender et al., 2021), the dynamic evolution of LLM architectures represents an additional source of uncertainty that challenges long-term reproducibility. Therefore, future replications in diverse domains and with newer model versions are essential to confirm the stability and transferability of the observed patterns.

6 Discussion

LLMs demonstrated considerable test case generation capabilities, achieving high precision. However, this result is accompanied by a consistent pattern of defects, notably incorrect facts, ambiguity, and inconsistency. This behavior is in line with recent literature on state-of-the-art models (Chen et al., 2024). The results suggest that while LLMs can significantly support Quality Assurance (QA) activities, their outputs still require careful and systematic human validation and complementary verification mechanisms to ensure reliability.

The hypothesis that the one-shot approach would outperform zero-shot in precision and defect reduction was not supported. As shown in Table 9, both prompting techniques exhibited comparable effectiveness, with one-shot achieving a marginal improvement in precision, but no statistically significant differences were found (all p-values > 0.05). Therefore, there is insufficient evidence to support a substantive quality gain from one-shot, suggesting that the observed effect may be attributed to natural variability in the runs.

Table 9. Comparison between zero-shot and one-shot Strategies

Metric	zero-shot	one-shot
Precision	0.718	0.741
F1 Score	0.618	0.587

This section discusses the empirical findings of our evaluation of LLM-generated test cases from real-world user stories, examining their strengths, limitations, and alignment with prior research. We interpret the observed defect patterns, the influence of prompting strategies, and the performance similarities across models, highlighting both methodological challenges and practical implications for QA workflows.

Comparison with oracle. Establishing rigorous semantic equivalence between generated test cases and the oracle proved complex. Typical performance metrics, while practical, may underestimate recall when semantically valid but non-identical tests appear. In this context, the manual classification of qualitative defects we introduced in this paper likewise introduces subjectivity to perform a more careful comparison with the oracle. Besides, it has proven useful to identify patterns of behavior, allowing the identification of the main limitations of the models and opportunities to improve prompting.

Our findings challenge the assumption of a perfect human oracle. While LLMs omitted a non-negligible portion of expected test cases, they also generated correct cases that were not present in the original human suite. This bidirectional omission pattern suggests that human and LLM generated test suites are complementary: models struggle with implicit, context-dependent behaviors, whereas humans may miss systematically derivable edge cases surfaced by prompt-driven generation.

AI-Generated “Innovative” Test Cases. The bidirectional omission pattern mentioned above reveals a critical finding: although LLMs omitted a significant portion of expected cases (FN: 508 zero-shot, 543 one-shot; Table 4, they also generated correct cases absent from the original oracle. Of the 291 correct cases with extraneous information

(19.06%; Table 5), approximately 12% were classified as genuine innovations by evaluators—systematic edge scenarios (e.g., *timeouts*, combinatorial validations) not contemplated in the human oracle.

This complementary generation demonstrates that LLMs can identify important test cases overlooked by the human oracle, expanding test coverage beyond the original analysts' perspective. However, 62% of these cases contained extraneous information (Table 4) requiring semantic filtering. Thus, the tester's role is redefined: from manual creator to curator of AI-generated artifacts.

Semantic defects and hallucinations. Approximately a quarter of the generated cases contained semantic defects, primarily incorrect facts analogous to hallucinations, suggesting that syntactically valid outputs can still misrepresent expected behavior. Our qualitative analysis revealed that these Incorrect Facts predominantly stem from the models' tendency to hallucinate domain-specific rules when faced with underspecified user stories. Because agile requirements often rely on the implicit, tacit knowledge of the development team, LLMs attempted to fill these contextual gaps with plausible yet factually incorrect system behaviors. To illustrate this phenomenon and our classification process, Table 10 provides a concrete, translated example of a correct versus a defective generated test case. In this sense, we understand that generated artifacts can serve as the first step in the test case generation process, followed by expert review. One possible way to improve this scenario is to incorporate automated or semi-automated quality verification steps in the pipeline. For instance, agent-based inspections would apply checklists to check the consistency between user stories and the generated test cases.

High incidence of omissions. Many omissions occurred when key contextual information was not explicit in the user story, causing the models to exclude relevant validation steps or acceptance criteria. We observed that this behavior was mainly due to the LLM's strict dependence on textual content and its limitations in contextual inference. After performing a deeper review of the oracle after the study execution, we identified that some test cases composing the oracle were mainly dependent on relevant information not available in the domain and user-story descriptions. In this sense, we confirmed that the test specialist who composed the original test cases often draws on experience and tacit knowledge to infer details that may not be explicitly documented.

Effects of prompting techniques. We found that the one-shot technique did not offer a statistically significant advantage over zero-shot, challenging the common premise of prompt engineering (Santana Jr et al., 2025). In addition, while one-shot tended to improve precision and reduce certain defects, zero-shot maintained a similar balance in the F1-score due to slightly higher recall. One possible explanation for this behavior remains open for further investigation.

While the combination of zero-shot (higher recall) and one-shot (higher precision) strategies suggests a promising direction for future work, this study was limited to the independent comparison of the two strategies as outlined in Section 3.5.

Concerns in selecting LLMs. The study findings suggest that the main challenges in selecting LLMs may be more

Table 10. Example of Generated Test Cases and Defect Classification (Translated from US01)

Description: As a writer of quotes and petitions, I would like them to be saved automatically to avoid data loss, for example, due to timeouts or connection problems. Acceptance Criterion: The system should provide functionality to automatically and periodically save user-entered data.
Generated Case A - CORRECT <i>Name:</i> CT03 - Continuity after connection loss <i>Pre-condition:</i> Authenticated user with a quote or initial petition being edited. <i>Objective:</i> Ensure that automatically saved data is not lost after a connection failure. <i>Expected Result:</i> Upon re-establishing the connection, the system allows the recovery of the automatically saved content. Classification: Correct
Generated Case B - DEFECTIVE <i>Name:</i> CT06 - Verify creation of movement linked to automatic saving <i>Pre-condition:</i> User creates a quote or initial petition. <i>Objective:</i> Validate that, when applicable, the system movement is created correctly with mandatory data. <i>Expected Result:</i> Movement related to the quote or petition is correctly created, with all mandatory data filled in. Classification: Defective (<i>Incorrect Fact</i>) <i>Reason:</i> Domain hallucination. In legal systems, a background auto-save to prevent data loss creates a draft; it must not generate a formal procedural "movement", which should only occur upon final submission.

strategic rather than technical. In this study, we focused on applying basic prompt settings and popular models. A performance plateau is suggested for test case generation from well-structured user stories. Unlike general benchmarks (Celik and Mahmoud, 2025), open-source models demonstrated comparable performance to state-of-the-art proprietary models. This implies that the choice of LLM may be guided by secondary factors such as cost and latency, rather than generation quality. Another relevant concern addresses data privacy and security, which may lead to opting out of local models. Therefore, we understand that optimizing LLM-based test generation should be treated as an ongoing process of continuous experimentation and quality control, rather than following a fixed approach.

Comparison with related work. Some of our findings are consistent with prior work, indicating that while LLM-generated tests are promising, they still exhibit substantial variability and require careful oversight. Notably, the high incidence of semantic defects, particularly incorrect facts, aligns closely with the factuality limitations and hallucination tendencies recently highlighted by Li et al. (2025) across broad software testing tasks. Furthermore, while previous literature in code-centric generation (e.g., Wang et al., 2024; Chen et al., 2024) demonstrates that LLMs struggle to achieve full coverage of complex control structures without specialized techniques, our results reveal a parallel challenge in natural language requirements: the systematic omission of implicit, context-dependent scenarios. Regarding prompting strategies, the absence of a statistically significant difference in overall effectiveness for the one-shot approach contrasts with earlier findings that reported clear gains when examples were included in prompts (Fan et al., 2023). This divergence suggests that improvements from example-based reasoning may be highly context-dependent. Specifically, the structured format of the user stories evaluated in this study, which already provides a high level of domain contextualization and organized acceptance criteria, likely supplies sufficient informational richness, thereby reducing the incremental impact of a single example. Finally, the comparatively lower recall observed in our real-world dataset, when contrasted with studies evaluating structural metrics or using synthetic benchmarks (such as those summarized by Mendoza et al. (2024), underscores the inherent complexity of industrial QA tasks and the limitations of evaluating LLMs solely through execution rates rather than semantic correctness.

Implications for practice. The results offer actionable insights for QA practice. LLM adoption reshapes the tester's role: from a creator of test cases to a curator and validator of AI-generated artifacts. This shift allows professionals to focus on critical analysis, coverage of complex scenarios, and domain-specific validation rather than repetitive manual tasks. Organizations can expect productivity gains, but human review must remain mandatory to prevent the propagation of defects such as incorrect facts and omissions. The observed functional equivalence between models supports flexibility in tool selection, enabling cost- and integration-based decisions rather than performance-driven ones. In agile contexts, LLMs should be integrated as assistive tools, complemented by automated post-processing mechanisms capable of detecting inconsistencies and ensuring the completeness

of generated test cases.

Paths for advancement: To strengthen LLM reliability in test generation, progress is required along three interconnected fronts. First, evaluation datasets should expand to include more diverse domains, enabling better generalization. Second, prompt engineering research should advance toward strategies explicitly designed to mitigate omissions and reasoning gaps. Finally, we understand that the semantic categorization applied in this study should be evolved to deeply assess test quality, revealing more specific challenges and limitations of the models. In this sense, we understand that collaboration between the NLP and Software Engineering communities is key to achieving these goals. Besides, longitudinal studies with evolving model versions can help track technological progress and identify best practices for integrating LLMs into QA pipelines (Santos et al., 2024).

7 Conclusion

This study investigated the effectiveness of Large Language Models (LLMs) for automatically generating functional test cases from user stories. Through a controlled evaluation involving three widely used models—GPT-4o, DeepSeek, and Gemini 1.5 Flash—under zero-shot and one-shot prompting conditions, we found that all evaluated models are capable of producing meaningful test cases. Despite this feasibility, our semantic analysis revealed recurring defects in the generated outputs, particularly incorrect facts, internal inconsistencies, ambiguities, and, most importantly, systematic omission of relevant test cases. These omissions were largely associated with implicit acceptance criteria, underscoring an inherent limitation of current LLM driven automation for test case generation.

This work makes three key contributions: (i) it provides a systematic, controlled evaluation of LLMs for generating complete and consistent test cases directly from user stories, addressing an emerging gap in automated software testing research; (ii) it introduces and applies a mixed set of evaluation criteria to assess the quality of LLM-generated test cases; and (iii) it delivers a controlled comparison between human-authored (oracle) and LLM-generated test sets, highlighting where automation offers measurable advantages and where human expertise remains essential.

Overall, our findings indicate that LLMs can serve as valuable assistants in the test case generation process, accelerating verification activities and reducing the likelihood of rework. In doing so, this study reinforces the view that LLMs have the potential to augment human expertise and improve productivity in software engineering practice.

Future research should explore more advanced prompting strategies, such as Retrieval-Augmented Generation (RAG) to provide richer contextual information and improve test coverage. A promising extension would be to combine zero-shot and one-shot in hybrid pipelines: generate test cases using both strategies simultaneously, remove duplicates via textual similarity or semantic analysis, and merge results to optimize global F1-score, leveraging the complementarity observed in the study results. Additionally, approaches for automated semantic validation of generated test cases are needed

to minimize reliance on manual review and strengthen the reliability of LLM-based test generation pipelines.

8 Data Availability

The replication package for this study, including the full prompts, the anonymized dataset of user stories, the complete set of generated test cases, and the data analysis scripts, is publicly available at: <https://github.com/leonardocesarc/testcase>.

References

- Bender, E. M., Gebru, T., McMillan-Major, A., and Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*, pages 610–623.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- Celik, A. and Mahmoud, Q. H. (2025). A review of large language models for automated test case generation. *Machine Learning and Knowledge Extraction*, 7(3):97.
- Chen, Y., Hu, Z., Zhi, C., Han, J., Deng, S., and Yin, J. (2024). Chatunitest: A framework for llm-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, pages 572–576.
- Fan, A., Gokkaya, B., Harman, M., Lyubarskiy, M., Sengupta, S., Yoo, S., and Zhang, J. M. (2023). Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, pages 31–53. IEEE.
- Fernández, D. M., Wagner, S., Kalinowski, M., Felderer, M., Mafra, P., Vetrò, A., Conte, T., Christiansson, M.-T., Greer, D., Lassenius, C., et al. (2017). Naming the pain in requirements engineering: Contemporary problems, causes, and effects in practice. *Empirical software engineering*, 22(5):2298–2338.
- Friedman, M. (1937). The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the american statistical association*, 32(200):675–701.
- Gheventer, A., Gurgel, P. d. A., Brito, C. H., de Mello, R. M., Rocha, S., Feitosa, R., and Travassos, G. H. (2026). Generative ai solutions for software quality: Assessing industrial readiness. *Software Quality Journal*. In Press.
- Li, Y., Liu, P., Wang, H., Chu, J., and Wong, W. E. (2025). Evaluating large language models for software testing. *Computer Standards & Interfaces*, 93:103942.
- Mendoza, I., Silva Filho, F., Medeiros, G., Paes, A., and Neves, V. O. (2024). Comparative analysis of large language model tools for automated test data generation from bdd. In *Simpósio Brasileiro de Engenharia de Software (SBES)*, pages 280–290. SBC.
- Myers, G. J., Badgett, T., Thomas, T. M., and Sandler, C. (2004). *The art of software testing*, volume 2. Wiley Online Library.
- Ouyang, S., Zhang, J. M., Harman, M., and Wang, M. (2025). An empirical study of the non-determinism of chatgpt in code generation. *ACM Transactions on Software Engineering and Methodology*, 34(2):1–28.
- Runeson, P., Host, M., Rainer, A., and Regnell, B. (2012). *Case study research in software engineering: Guidelines and examples*. John Wiley & Sons.
- Santana Jr, E., Benjamin, G., Araujo, M., Santos, H., Freitas, D., Almeida, E., Neto, P. A. d., Li, J., Chun, J., and Ahmed, I. (2025). Which prompting technique should i use? an empirical investigation of prompting techniques for software engineering tasks. *arXiv preprint arXiv:2506.05614*.
- Santos, S. R. R. d., Fernandes, R. E. S., Santos, M. C. B. d., Soares, M. S., Rocha, F. G., and Marczak, S. (2024). Increasing test coverage by automating bdd tests in proofs of concepts (pocs) using llm. In *Proceedings of the XXIII Brazilian Symposium on Software Quality*, pages 519–525.
- Schäfer, M., Nadi, S., Eghbali, A., and Tip, F. (2023). An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 50(1):85–105.
- Travassos, G., Shull, F., Fredericks, M., and Basili, V. R. (1999). Detecting defects in object-oriented designs: using reading techniques to increase software quality. *ACM sigplan notices*, 34(10):47–56.
- Travassos, G. H., Gurov, D., and Amaral, E. (2002). *Introduction to Experimental Software Engineering*, volume 9. UFRJ Rio de Janeiro, Brazil.
- Verdecchia, R., Engström, E., Lago, P., Runeson, P., and Song, Q. (2023). Threats to validity in software engineering research: A critical reflection. *Information and Software Technology*, 164:107329.
- Verkholomova, A. and Nazaruka, E. (2025). Overview of the application of generative ai in software performance testing. In *2025 66th International Scientific Conference on Information Technology and Management Science of Riga Technical University (ITMS)*, pages 1–6. IEEE.
- Wang, Z., Liu, K., Li, G., and Jin, Z. (2024). Hits: High-coverage llm-based unit test generation via method slicing. pages 1258–1268.
- White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., and Schmidt, D. C. (2023). A prompt pattern catalog to enhance prompt engineering with chatgpt. *arXiv preprint arXiv:2302.11382*.
- Whittaker, J. A. (2002). *How to break software: A practical guide to testing with Cdrom*. Addison-Wesley Longman Publishing Co., Inc.
- Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics bulletin*, 1(6):80–83.
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., and Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer.